Handoff 01: ENG-01 content format and result states

Date: 2026-10-24 (America/Toronto)
Written by: Claude Code (implementer)
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: pull request open, not merged, not deployed

> **How to use this file:** paste or upload it at the start of a chat session together with `build.md`. Treat it as a summary of evidence, not proof: the pull request diff and CI results are the source of truth (`build.md` section 14).

Starter prompt for the chat assistant

```
You are helping me continue Python Academy, which is expanding into a broader
computing learning platform. I'm attaching build.md and this handoff. Treat
roadmap items as proposals unless this handoff or the code shows they are
implemented.

Your role this session is: reviewer. Review pull request #5 (ENG-01) against
the acceptance criteria below. Look for gaps in testing, places where
existing behaviour could have changed, and anything that conflicts with
build.md. Explain at a first-year computer engineering level. Do not claim
anything is live or tested unless this handoff shows the evidence.
```

1. Project in one paragraph

Python Academy is a learning platform: a Next.js/React/TypeScript frontend and a FastAPI/SQLAlchemy backend on PostgreSQL, deployed on Render, with Google sign-in. Parents (over 25) invite learners (13 or older). Today there is one chapter, "Build Your Own Adventure": five quests, each with five cards shown one at a time (discover, see it work, predict, try it, your challenge). Learner Python runs **only in the browser** (Pyodide in a web worker). The server never runs learner code; it checks a small, explicitly supported subset of Python by reading the code's syntax tree. Repository: `torontojohn/python-academy` (private).

2. Where we are in the plan

Stage	What	Status
Plan	`docs/plan/course-catalog.md`: catalog, goals, recommendations, Python course outline, XP/badges, decisions	Committed on branch `plan/course-catalog` (commits `5466166`, `46e9559`); **not pushed yet**
1. ENG-01	Content format and result states	**in_review**: PR #5
2. ENG-02	Alembic migrations, then XP and badge ledgers with a backfill	Not started; waits for maintainer OK on PR #5
3. Catalog UI	Catalog page, course page, lesson page with unit sidebar	Not started
4. Goals and recommendations	Goal card at sign-up, rule-based recommendation function with tests	Not started

Rules for every stage (from the maintainer): one branch and pull request per stage; run existing and new tests and report exact commands; never merge to `main` or deploy; no secrets, no new paid services, no change to age rules or account policy; never run learner code on the server; stop after each stage for review.

Maintainer decisions recorded in the plan (2026-10-24)

1. One primary learner goal for the pilot.
2. Parents see a learner's goal and completed courses, not placement answers.
3. Next Python unit to build after groundwork: unit 2, Numbers and decisions.
4. Checking for units 2+: browser behaviour checks (the Pyodide worker runs learner code against test cases from the content file); keep the existing bounded checker for unit 1.
5. The five existing lesson IDs stay permanently, with no aliases.
6. XP: 100 per lesson challenge, 200 per unit project, 500 per course final project; stretch work earns a badge only.
7. Regular expressions are a core Python unit, skippable via placement.
8. Unreleased courses appear in the catalog labelled "Coming soon" and cannot be started.
9. Hackathon goal path: Python units 1 to 7, then `hack`, then branch.
10. New course `hack` (Hackathon Ready): requires Python units 1 to 7; suggests `git` and `web`.

3. Handoff record (build.md template)

Field	Value
Task / issue	ENG-01: versioned content format and result states (`build.md` backlog; plan section 7.1 and 7.2)
Human goal	Prepare the content and grading groundwork for a multi-course catalog without changing anything learners see
Relevant build.md sections	4 (teaching contract), 11 (content model, result states), 12 (per-feature gate), 13 (backlog)

Field	Value
Repository and branch	`torontojohn/python-academy`, branch `eng-01/content-format`
Base commit / head commit	Base `a293798` (`main`) / head `4a86973` (first round `bd592f4`, review fix `4a86973`)
Pull request	https://github.com/torontojohn/python-academy/pull/5
Implementer / reviewer	Claude Code / maintainer, plus chat assistant if wanted
Deployment status	Not requested. Not merged, not deployed

Files changed and why

File	Change
`backend/content/catalog.json`	New. All 11 planned courses and their units in display order. Only `py.u01` is released; everything else is `coming_soon`
`backend/content/py/u01/*.json`	New. One file per existing lesson (`hello`, `variables`, `input`, `choices`, `adventure`) with five typed tasks, IDs, versions, badge record, and validator name. Text copied unchanged by script
`backend/content/README.md`	New. Explains the format for contributors
`backend/app/content.py`	New. Schema (Pydantic), load-time validation, and a function that rebuilds the original `/api/lessons` shape
`backend/app/lesson_checks.py`	Adds result states; marks checker-limit messages as `unsupported`; catches unexpected failures as `service_error`
`backend/app/main.py`	Loads lessons from the new content; `/api/check` and `/api/attempts` return `result` next to `passed`
`backend/tests/fixtures/legacy_lessons_v1.json`	The old `curriculum.json`, moved unchanged and frozen as the "exactly as before" reference
`backend/tests/test_content.py`	New. 27 tests, including one proving learner code never reaches the server log
`backend/tests/test_academy.py`	2 new API tests
`frontend/tests/academy.spec.ts`	Imports lessons from the frozen fixture (the old path no longer exists); new test for what a learner sees on `service_error`
`build.md`	Source-locations table updated to the new paths

Acceptance criteria

- The five existing lessons work exactly as before. **Met**: `/api/lessons` returns identical JSON; all browser tests pass; real-stack run passed all five.
- Content has stable IDs and versions, organised as course > unit > lesson > task. **Met**.
- Content is validated: references resolve, no prerequisite cycles, card sequence, known validators. **Met**: one failing-content test per rule.
- Results distinguish learner mistakes from checker limits and service failures. **Met** for `passed`, `needs_work`, `unsupported`, `service_error`. `execution_error` is defined but not produced yet (browser runtime errors stay in the browser today).

Non-goals

No UI changes, no database or schema changes, no new endpoints, no new lessons, no change to account or age rules.

Behaviour before and after

Area	Before	After
Lesson text, cards, UI	—	Unchanged
`/api/lessons`	Read from `curriculum.json`	Rebuilt from content files; identical JSON
Check response	`passed`, `feedback`, `assessment`	Same, plus `result`
Feedback wording	—	Unchanged
Checker crash	HTTP 500; UI showed "Your practice could not be checked"	`result: service_error`; learner sees "Unable to assess this right now. Your code is unchanged; please try again in a moment." No badge or XP; code kept; can retry. Server logs only lesson ID, error type, and server code location, never learner code
Invalid content	Not detected	API refuses to start and lists every problem

Result states

Result	Meaning	Examples
`passed`	Meets the lesson's checks	Two nonempty print statements in `hello`
`needs_work`	Something the learner can fix	Syntax error, undefined variable, missing player reply, wrong output, hard-coded name

Result	Meaning	Examples
`unsupported`	Valid work this checker cannot judge	`import`, loops, adding numbers, size or step limits
`service_error`	The check itself failed	Unexpected exception in the checker
`execution_error`	The program failed while running in the browser	Reserved for later stages

Tests run

Local machine: Python 3.12.14 (via uv, `backend/.venv`), Node 24.19.0, Chromium headless shell 156.

Command	Before, on `main`	After, on `4a86973`
`cd backend && python -m pytest -q`	42 passed	**71 passed**
`cd frontend && npm ci --include=dev && npm run build`	Succeeded	**Succeeded**
`cd frontend && npm run test:e2e`	14 passed	**15 passed** (one new)
GitHub Actions on PR #5	—	**backend pass, frontend pass** (runs on `bd592f4` and `4a86973`)

Extra checks:

- Changing one character in a lesson file makes both equality tests fail (then reverted).
- Real stack, not mocked: uvicorn plus `next start` with a temporary SQLite database. `/api/lessons` through the Next.js proxy was identical to the original. In Chromium, every lesson's example ran in real Pyodide and the real `/api/check` returned `passed`. An `import` returned `unsupported` with the original feedback. At 375px wide with reduced motion there was no horizontal scroll.

Mocks and unverified behaviour

- Browser tests mock the account and check API responses; Python execution in them is real.
- Not verified: Render deployment, PostgreSQL (no schema change in this stage), production Google sign-in.

Screenshots

Taken against the real local backend; they match the existing design, which is the intended result. Stored outside the repository on the maintainer's machine; ask the maintainer if you need them.

Data migration and compatibility

None. `attempts.lesson_id` values are unchanged because lesson IDs are unchanged. The `result` is not stored yet; that needs ENG-02 migrations.

Known limitations and open questions

- The UI still uses the legacy `/api/lessons` shape. Typed catalog endpoints and frontend types come in stage 3.
- `backend/content/README.md` links to `docs/plan/course-catalog.md`, which lands when the plan branch is pushed and merged. Suggested order: merge the plan first.
- Should the plan branch get its own pull request now? (Maintainer to decide.)

Review follow-up (round 1)

A chat reviewer asked two questions before approving.

1. **Does content validation run in CI?** Yes. GitHub's backend job runs `python -m pytest -q` (71 passed on PR #5). Every backend test file imports the app, and importing the app loads and validates all content, so a bad lesson file fails CI. Branch protection to require passing checks before merging must be turned on by the repository owner (`torontojohn`); on a private repository it may need a paid GitHub plan. If it is unavailable, the free rule is: merge only when the pull request shows all checks green.
2. **What does a learner see on `service_error`, and is learner code logged?** The learner sees the "Unable to assess" message in the feedback area, gets no badge or XP, keeps their code, and can run again (new browser test). The first version **did** risk logging learner code: it used `logging.exception`, whose message and traceback can echo learner text. Fixed in `4a86973` to log only lesson ID, error type, and server code location. A new test fails on the old code (it captured `RuntimeError: checker bug near learner-private-story-text`) and passes on the fix.

Merge order: plan pull request first, then PR #5. Merging into `main` deploys to Render, so check the live site afterwards.

Next concrete action

Maintainer reviews PR #5. On approval, start ENG-02 on a new branch: Alembic baseline matching today's tables, then XP and badge award tables with unique keys and a backfill from existing passing attempts, with migration tests from existing accounts.

4. Earlier context: security review (2026-10-24, before this work)

No API keys or secrets were found in the code or in the full git history. Main risks to address before broader use: no rate limits or quotas; sessions are signed cookies, so logout cannot revoke a stolen cookie for up to 24 hours; the API service has its own public URL; the main pages have no Content-Security-Policy or HSTS header; Pyodide loads from a CDN without integrity checks; invitation tokens appear in URLs. None of these changed in this stage.